

DIAPALER AFRICA — Ce qu'on a fait, en mots simples

✦✦ Ce qui marche déjà dans l'application

L'écran d'accueil

Quand on lance l'application, on voit d'abord un petit film d'animation : le logo (deux mains qui se serrent — ça veut dire "s'épauler" en wolof) apparaît, et trois petits points aux couleurs du drapeau du Sénégal (vert, jaune, rouge) viennent l'entourer. C'est très court, ça dure une seconde, juste pour faire jolie.

Choisir qui on est

Ensuite, l'application demande : « Qui es-tu ? ». Trois choix possibles : **Entrepreneur**, **Mentor** ou **Investisseur**. Pour l'instant, seuls les **entrepreneurs** peuvent vraiment s'inscrire. Pour les mentors et investisseurs, l'écran reste vide en attendant qu'on développe leurs versions.

S'inscrire (pour les entrepreneurs)

L'inscription se fait en **4 étapes simples**, pour ne pas tout demander d'un coup :

1. **Qui je suis** : nom complet, email, sexe, date de naissance. Quand on tape sa date, l'âge s'affiche automatiquement à côté ! Si on tape un email mal écrit, un petit signal rouge apparaît tout de suite. Si c'est correct, c'est un signal vert.
2. **D'où je viens** : on choisit son pays (Sénégal, Gambie ou Mali), puis sa ville dans une liste qui s'adapte au pays choisi, et on peut ajouter son adresse.
3. **Pour mieux me connaître** : on peut écrire une petite présentation de soi, mettre son lien LinkedIn, et **on doit obligatoirement choisir au moins un domaine qui nous intéresse** (mode, agriculture, technologie, etc. — il y en a plus de 30 disponibles).
4. **Sécurité** : numéro de téléphone (avec le drapeau du Sénégal et l'indicatif +221 déjà mis), un mot de passe (avec une petite barre qui montre si c'est faible, moyen ou fort) et accepter les conditions d'utilisation.

À chaque étape, le bouton « Continuer » reste gris tant qu'on n'a pas tout bien rempli — comme ça on ne peut pas avancer avec des informations manquantes.

La page d'accueil après connexion

Une fois connecté, on tombe sur un tableau de bord personnalisé qui salue par notre prénom : « **Bonjour, Marième** 🌸 ». Sous le bonjour, des citations inspirantes tournent automatiquement toutes les 6 secondes (proverbes wolof, citations d'entrepreneurs africains comme Aliko Dangote ou Magatte Wade).

On y voit :

- **Notre projet en cours** dans une grande carte bleue avec une barre de progression qui s'anime (par exemple « 60 % terminé »). Si on n'a pas encore de projet, à la place, on voit un gros bouton « + » très visible qui invite à créer son premier projet.
- **Nos statistiques** : nombre de mentors avec qui on est en contact, sessions à venir, pitches déposés, favoris. Tout est à zéro pour un nouveau compte (logique !).
- **Mentors recommandés** : selon les domaines qu'on a choisis, l'application affiche **les 2 mentors les plus pertinents pour nous**. Si on change ses centres d'intérêt plus tard, les recommandations changent automatiquement.
- Un raccourci vers les programmes de financement public (DER/FJ, programme PAVIE 2 avec ses 107 milliards de FCFA).

□ Trouver un mentor

Sur la page « Matching », on a accès à **12 mentors sénégalais réels** (Anta Dama Kama, Yérém Habib Sow, Aïssa Dione, Pape Diouf de Wave, etc.). Pour chacun :

- Une **carte avec sa photo, son métier, sa note (4.8 ★) et son score de compatibilité avec nous**
- Un **badge CIS** (Club des Investisseurs Sénégalais) pour les membres
- On peut **chercher** par nom, secteur ou ville
- On peut **filtrer** par secteur (mode, tech, finance, automobile, énergie...) ou par ville

Quand on clique sur un mentor, on voit sa **fiche détaillée** : son parcours, ses domaines d'expertise, **toutes ses entreprises** (Yérém Sow par exemple en a 6 listées, Mansour Ndao en a 8), et des créneaux pour réserver une session.

🚀 Déposer son projet (pitch)

Au milieu de la barre du bas, il y a un **gros bouton rond doré avec un « + »**. Quand on appuie dessus, on accède à un formulaire en 3 étapes pour présenter son projet : titre, secteur d'activité, description, montant souhaité, et zones pour ajouter un PDF de présentation et une vidéo.

👤 Notre profil

On peut accéder à son profil en tapant sur sa photo en haut à gauche. On y voit :

- Photo, nom, statut « en ligne » (point vert)
- Toutes nos infos : email, téléphone, sexe, date de naissance avec l'âge calculé, adresse, ville, pays
- Notre LinkedIn si on l'a renseigné
- La **liste de tous nos projets** (ceux en cours et ceux terminés)
- Nos centres d'intérêt
- Notre bio

Un bouton « Modifier mon profil » permet de tout changer. Toutes les modifications sont enregistrées immédiatement et synchronisées en ligne.

Sécurité et données

Toutes les inscriptions et modifications sont **vraiment sauvegardées sur internet** (on utilise le service Firebase de Google). Si on se déconnecte et qu'on revient plus tard, on retrouve son compte et toutes ses informations. Plusieurs personnes peuvent utiliser l'application en même temps, chacune avec son compte.

Petits détails qui font la différence

- **Sur ordinateur**, des **petites étoiles aux couleurs du drapeau sénégalais** suivent la souris partout dans l'application — effet magique
- **Au survol** d'une carte (mentor, projet, statistique), elle se soulève légèrement avec une lueur dorée
- Les **chiffres s'animent** depuis 0 quand on ouvre une page (le « 60 % » du projet, par exemple)
- **Toutes les pages sont fluides** avec des transitions douces

Ce qu'il reste à faire

● Le plus urgent (à faire en priorité)

Sécurité du compte

- **Récupérer son mot de passe oublié** : actuellement le lien existe mais ne fait rien. Il faut qu'on puisse recevoir un email pour le réinitialiser.
- **Vérifier son adresse email** : qu'on doive cliquer sur un lien envoyé par email pour confirmer qu'elle nous appartient.
- **Code par SMS (OTP)** : pour s'inscrire avec son numéro de téléphone, recevoir un code à 6 chiffres et le valider.
- **Supprimer son compte** : obligation légale (loi sénégalaise CDP). Permettre à l'utilisateur de tout effacer s'il le souhaite.

Mentor et Investisseur

- **Faire l'inscription** pour ces deux profils (qui pour l'instant ne peuvent pas s'inscrire)
- **Créer leurs tableaux de bord dédiés** :
 - Le mentor verrait ses élèves, ses demandes en attente, ses sessions à venir, les pitches qu'il a reçus
 - L'investisseur verrait les pitches intéressants, son portefeuille d'investissements, ses retours

Communication

- **Une vraie messagerie** entre entrepreneur et mentor (pour l'instant le bouton existe mais ne mène nulle part)

- **Notifications** : recevoir une alerte quand un mentor répond, quand une session approche, quand son pitch est consulté

□ Important (à faire ensuite)

Améliorer le pitch et les projets

- **Sauvegarder les pitches** déposés (pour l'instant le formulaire existe mais le pitch n'est pas réellement enregistré)
- **Pouvoir modifier ou supprimer** un projet déjà créé (pour l'instant on peut juste le créer)
- **Téléverser sa vraie photo** de profil (pour l'instant ce sont juste les initiales sur fond doré)
- **Marquer des mentors en favoris** avec un cœur ou un signet
- **Réserver une vraie session** avec un calendrier qui marche

Lien avec l'écosystème sénégalais

- **Module DER/FJ complet** : afficher les vraies fiches des programmes (PAVIE 2, Be Yes), aider à monter son dossier de candidature
- **Vue calendrier** des sessions : ce qui est passé, ce qui arrive
- **Bibliothèque de ressources** : articles, modèles de business plan, mini-formations en français et en wolof
- **Donner une note** au mentor après une session (et que cette note influence les futures recommandations)

Aspects légaux

- Vraies pages de **Conditions d'Utilisation** et **Politique de Confidentialité**

□ Ce serait bien d'avoir (plus tard)

Téléphones et publication

- **Faire fonctionner l'app sur Android** (pour l'instant elle ne marche que dans le navigateur Chrome)
- **Mettre l'app sur le Play Store** et l'App Store
- **Publier la version web** sur internet pour que tout le monde puisse y accéder sans installation

Confort et professionnalisme

- **Mode sombre** pour ceux qui préfèrent
- **App en wolof** en plus du français
- **Mode hors-ligne** : pouvoir consulter les mentors et conversations même sans internet
- **Adapter l'écran aux tablettes**
- **Accessibilité** pour les personnes malvoyantes (lecteur d'écran, gros caractères, contrastes)

✳ Vision long terme

- **Algorithme intelligent de matching** qui apprend des préférences (intelligence artificielle)
 - **Paielement par mobile money** (Wave, Orange Money) pour payer les sessions
 - **Vidéoconférence directement dans l'app** (comme Zoom mais intégré)
 - **Programme de parrainage** : on invite un ami, on reçoit une récompense
 - **Modération** : possibilité de signaler ou bloquer un utilisateur abusif
 - **Aide et FAQ** dans l'app
 - **Page À propos** et **Contact**
-

🎯 En résumé

L'application est **fonctionnelle aujourd'hui** : on peut s'inscrire en vrai, se connecter, modifier son profil, créer des projets, voir des mentors recommandés selon ses goûts, et le tout est sauvegardé en ligne et synchronisé. C'est déjà une vraie application qui marche.

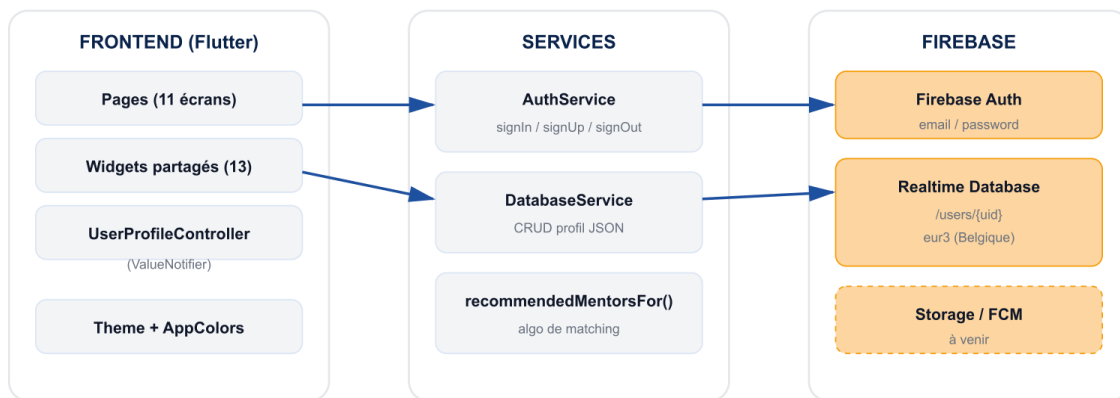
Mais c'est **un début** : il reste beaucoup à faire pour qu'elle soit complètement utile à un entrepreneur sénégalais — surtout la messagerie, les comptes Mentor/Investisseur et la liaison avec les programmes publics comme la DER/FJ. Le travail réalisé constitue une **base solide** sur laquelle construire la suite.

RAPPORT TECHNIQUE SUR CE QU'ON A FAIT

2.1 Architecture client / serveur

L'app Flutter (web Chrome pour la démo, Android prévu Livrable 1) communique avec deux services Firebase. La logique d'accès aux services est isolée dans des wrappers (services/) pour pouvoir évoluer ou tester indépendamment de l'UI.

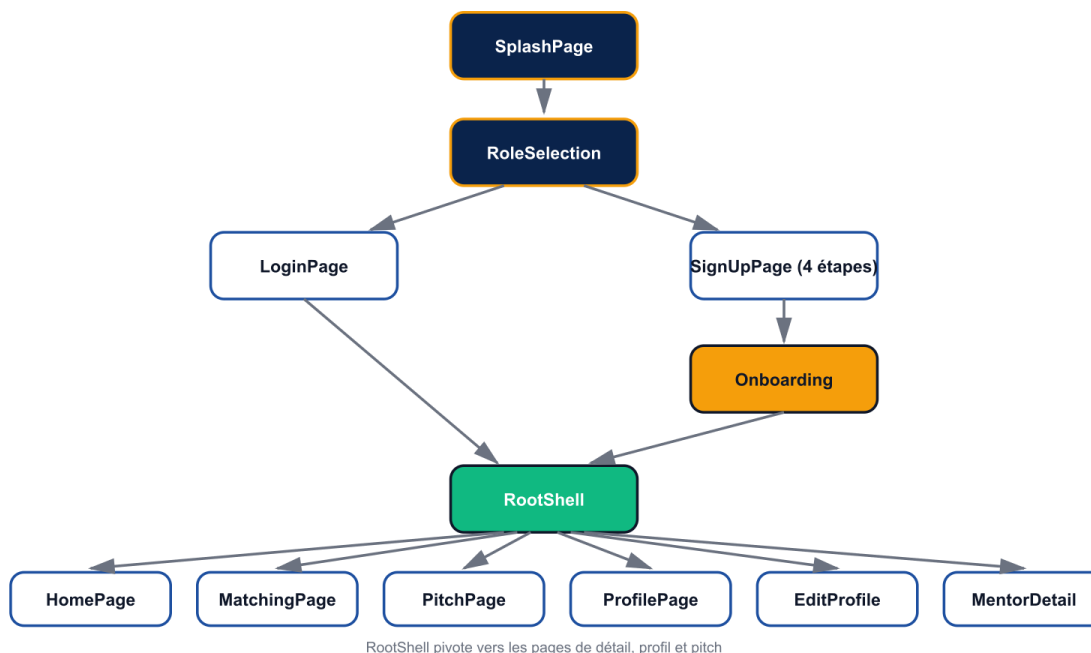
Architecture client / serveur



2.2 Flow de navigation des écrans

Tous les écrans sont accessibles depuis le RootShell (bottom nav 2 onglets + FAB central pour le pitch). Le splash effectue une auto-login si la session est active, sinon route vers la sélection de rôle.

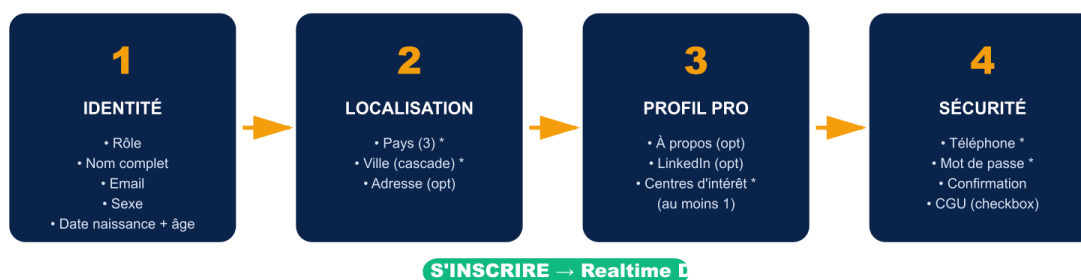
Flow de navigation des écrans



2.3 Inscription en 4 étapes

Pour réduire la charge cognitive, l'inscription est découpée en 4 sections thématiques. Chaque étape valide localement avant de débloquent la suivante. Centres d'intérêt obligatoires (≥ 1) pour permettre le matching personnalisé.

Inscription en 4 étapes



2.4 Algorithme de recommandation

Les mentors recommandés sur le dashboard sont calculés en live à partir de trois sources combinées dans le profil utilisateur : le secteur déclaré, les secteurs des projets actifs et les centres d'intérêt.

Algorithme de recommandation des mentors



3. Stack technique

Couche	Technologie
Framework UI	Flutter 3.41 / Dart 3.11
Plateforme cible	Web (Chrome) — Android prévu Livrable 1
Authentification	Firebase Auth (email/password)
Base de données	Firebase Realtime Database (eur3 / Belgique)
Polices	Inter (via google_fonts)
État de l'app	ValueNotifier (UserProfileController) — pas de Provider/Bloc
Versionnement	Git + GitHub avec branch protection sur main
Workflow	Feature branches + Pull Requests + review owner
Build mode démo	flutter run -d chrome --release

□ Choix d'architecture

Pas de Provider, pas de Bloc, pas de GetX. On utilise les ValueNotifier natifs Flutter pour partager l'état (UserProfileController). Pour un projet de cette taille, c'est suffisant et garde le code lisible. À reconsidérer si on dépasse 30+ écrans.

4. Structure du code

Tout est dans le dossier lib/. Pas de sur-engineering : un fichier par écran, un fichier par widget réutilisable. Les services Firebase sont isolés dans services/.

```
lib/ └─ main.dart                               Entry point + Firebase init parallèle au
splash └─ firebase_options.dart                 Config Firebase Web (généré manuellement)
└─ theme/ └─ app_theme.dart                     AppColors (drapeau Sénégal) +
ThemeData light └─ data/ └─ user_profile.dart    Modèle UserProfile,
Project, Gender + Controller └─ mock_data.dart    12 mentors
sénégalais + recommendedMentorsFor() └─ countries.dart    Pays/villes
(Sénégal/Gambie/Mali) └─ quotes.dart              8 citations (proverbes
wolof + entrepreneurs) └─ services/ └─ auth_service.dart  Wrapper
FirebaseAuth + erreurs FR └─ database_service.dart  CRUD profil sur
Realtime Database └─ screens/                    11 pages └─
splash_page.dart └─ role_selection_page.dart └─ login_page.dart └─
signup_page.dart └─ onboarding_page.dart └─ root_shell.dart └─
home_page.dart └─ matching_page.dart └─ mentor_detail_page.dart └─
profile_page.dart └─ edit_profile_page.dart └─ add_project_page.dart └─
└─ pitch_page.dart └─ widgets/                  13 widgets partagés └─
animated_counter.dart └─ avatar.dart └─ bottom_nav.dart └─
cursor_follower.dart └─ diapaler_logo.dart └─ flag_strip.dart └─
hover_glow_card.dart └─ mentor_card.dart └─ profile_sheet.dart └─
quote_carousel.dart └─ rotating_tagline.dart └─ section_header.dart
└─ skeleton.dart
```

5. Fonctionnalités livrées

Pour chaque fonctionnalité ci-dessous, on précise ce qu'elle fait, le ou les fichiers concernés, et les principales fonctions ou classes à connaître pour intervenir dessus.

5.1 Authentification Firebase

Toute la logique d'authentification est isolée dans un service dédié qui wrap

`FirebaseAuth.instance`. L'app n'appelle jamais directement Firebase — elle passe toujours par `AuthService`. Les méthodes `signIn()`, `signUp()` et `signOut()` retournent les `UserCredential` standards. La méthode `humanError()` traduit les 12 codes d'erreur Firebase en français lisible (« Email ou mot de passe incorrect », « Mot de passe trop faible », etc.).

Au moment de l'inscription, on crée immédiatement le node `/users/{uid}` dans la Realtime Database via `DatabaseService.createUserProfile()`. À chaque connexion,

`LoginPage._signIn()` recharge le profil distant et appelle `UserProfileController.update()` pour rafraîchir l'UI globalement.

Fichiers : `lib/services/auth_service.dart` · `lib/services/database_service.dart` · `lib/firebase_options.dart` · `lib/main.dart`

5.2 Auto-login au démarrage

Le splash sert aussi de gate d'authentification. Dans `main.dart`, la variable globale `firebaseReady` lance `Firebase.initializeApp()` en parallèle de `runApp()` — l'animation tourne pendant l'init. La méthode `SplashPage._bootstrap()` attend Firebase puis vérifie `AuthService.currentUid` : si la session est active, on lit le profil distant via `DatabaseService.readUserProfile(uid)` et on route directement vers `RootShell`. Sinon on tombe sur `RoleSelectionPage`.

Fichiers : `lib/main.dart` · `lib/screens/splash_page.dart`

5.3 Inscription Entrepreneur en 4 étapes

Toute l'inscription est dans un seul `StatefulWidget` — `SignUpPage` — qui maintient un compteur `_step` (0..3) et 4 builders distincts (`_buildStep1` à `_buildStep4`). Chaque étape a son getter de validation : `_step1Valid`, `_step2Valid`, `_step3Valid`, `_step4Valid`. Le bouton CONTINUER appelle `_next()` qui incrémente `_step` ou déclenche `_submit()` sur la dernière étape.

Les validations live utilisent des regex (`_emailRegex`), un calcul de force de mot de passe (`_computeStrength()` → 0..4 affiché par `_StrengthMeter`), et un formater de téléphone custom (`_PhoneFormatter`) qui transforme 771234567 en 77 123 45 67 automatiquement.

Les pays et villes en cascade viennent de `data/countries.dart` : 3 pays (Sénégal, Gambie, Mali) avec leurs villes. Quand l'utilisateur change le pays, on appelle `citiesOf(country).first` pour reset la ville. La date de naissance ouvre un `showDatePicker()` et l'âge live est calculé par `ageFromBirthDate(_birthDate)`.

Pour Mentor et Investisseur, le formulaire est volontairement vide — on affiche `SnackBar.shrink()` à la place du contenu. Le bouton CONTINUER reste désactivé tant que `_role != UserRole.entrepreneur`.

Fichiers : `lib/screens/signup_page.dart` · `lib/data/countries.dart`

5.4 Profil utilisateur partagé en mémoire

L'état du profil est centralisé dans `UserProfileController.profile` — un `ValueNotifier<UserProfile>` unique pour toute l'app. Toutes les vues qui affichent ou modifient le profil l'écoutent via `ValueListenableBuilder<UserProfile>` : `HomePage._Header`, `ProfilePage`, `ProfileSheet`, `EditProfilePage`. Quand on appelle `UserProfileController.update(next)`, toutes ces vues se rebuild instantanément.

Le modèle `UserProfile` est `@immutable` avec un `copyWith()` complet. Les champs : `firstName`, `lastName`, `email`, `phone`, `gender`, `birthDate`, `address`, `city`, `country`, `sector`, `role`, `bio`, `linkedin`, `interests`, `projects`, `mentorsActive`, `sessionsCount`, `favoritesCount`, `score`.

Fichiers : `lib/data/user_profile.dart`

5.5 Persistance dans la Realtime Database

Les méthodes CRUD du profil sont dans `DatabaseService` : `createUserProfile(uid, profile)`, `updateUserProfile(uid, profile)`, `readUserProfile(uid)`. La sérialisation se fait via les helpers privés `_toMap(p)` et `_fromMap(m)` qui gèrent la conversion `Gender` ↔ `String`, `DateTime` ↔ `ISO 8601`, et la liste imbriquée des projets.

À chaque sauvegarde, on ajoute `ServerValue.timestamp` sur le champ `updatedAt` — utile pour les futurs sync conflicts.

```
/users/{uid}  |— firstName, lastName, email, phone  |— gender, birthDate,
address, city, country  |— sector, role, bio, linkedin  |— interests:
[String]  |— projects: [{ id, name, description, sector, step, totalSteps }]
|— mentorsActive, sessionsCount, favoritesCount, score  |— updatedAt
(ServerValue.timestamp)
```

Fichiers : `lib/services/database_service.dart`

5.6 Multi-projets avec règle « 1 actif à la fois »

Le modèle `Project` est défini dans `user_profile.dart` avec ses helpers `isCompleted` (`step ≥ totalSteps`) et `progress` (`step / totalSteps`). La règle métier est centralisée dans le getter `UserProfile.canStartNewProject` : il retourne `true` uniquement si tous les projets existants sont terminés (ou s'il n'y en a aucun).

L'ajout passe par `UserProfileController.addProject(p)` qui vérifie d'abord la règle. La page `AddProjectPage` présente un formulaire simple (nom + secteur + description) avec validation.

Le bouton « + Nouveau projet » sur `ProfilePage` est grisé via `_NewProjectButton(canStart: profile.canStartNewProject)`.

Quand l'utilisateur n'a aucun projet, le hero du dashboard bascule en mode empty state via `_EmptyProjectHero` — un gros bouton + ambre cliquable qui ouvre `AddProjectPage`.

Fichiers : `lib/data/user_profile.dart` · `lib/screens/profile_page.dart` ·
`lib/screens/add_project_page.dart` · `lib/screens/home_page.dart`

5.7 Dashboard Entrepreneur

Le dashboard `HomePage` est un `StatefulWidget` avec un flag `_loading` qui affiche les skeletons shimmer pendant 900 ms au boot. Le `RefreshIndicator` déclenche `_refresh()` qui simule un reload (700 ms).

Tous les sous-blocs réagissent au profile : `_Header` (greeting + avatar + tagline), `_ProjectHero` (bascule entre la card projet et l'empty state), `_StatsStrip` (5 mini-cards lues du profile via `ValueListenableBuilder`), et `_RecommendedMentors` (top 2 mentors filtrés en live).

Le greeting personnalisé utilise `p.firstName`. La tagline rotative `RotatingTagline` est un widget qui alterne les 8 citations toutes les 6 secondes via un `Timer.periodic`.

Fichiers : `lib/screens/home_page.dart` · `lib/widgets/rotating_tagline.dart` ·
`lib/widgets/skeleton.dart`

5.8 Algorithme de recommandation

La fonction `recommendedMentorsFor()` dans `data/mock_data.dart` prend trois sources de secteurs : `userSector`, `userInterests`, `projectSectors`. Elle calcule pour chaque mentor le nombre de secteurs en commun (`overlap`), filtre ceux avec `overlap > 0`, et trie d'abord par `overlap` décroissant puis par `compatibility`.

Le widget `_RecommendedMentors` (dans `home_page.dart`) l'appelle dans un `ValueListenableBuilder` : dès que l'utilisateur modifie ses centres d'intérêt dans `EditProfilePage`, le dashboard se rafraîchit avec les nouvelles recommandations.

Fichiers : `lib/data/mock_data.dart` · `lib/screens/home_page.dart`

5.9 Matching de mentors

La page `MatchingPage` contient les 12 mentors hardcodés dans `mock_data.dart`. Le filtrage combine recherche texte (`Mentor.matches(query)` — case-insensitive sur nom, secteurs, ville, titre), un filtre de secteur (pills horizontales) et un filtre de ville (`DropDownButton`). Le getter `_filtered` applique les 3 filtres et trie par compatibility.

Au tap sur une carte, `MentorCard` ouvre `MentorDetailPage` — un `CustomScrollView` avec `SliverAppBar` gradient navy. La page affiche 4 stats (note, match, années, avis), les domaines, la liste complète des entreprises (`_CompaniesList`), les créneaux (`_SlotsRow`), et 2 CTA (Message / Réserver).

Fichiers : `lib/screens/matching_page.dart` · `lib/screens/mentor_detail_page.dart` · `lib/widgets/mentor_card.dart` · `lib/data/mock_data.dart`

5.10 Dépôt de pitch

Accessible via le `FloatingActionButton` ambre central de `RootShell` (placement `centerDocked`). La page `PitchPage` utilise un compteur `_step` (0..2) avec `_buildStep1` (titre + elevator pitch), `_buildStep2` (secteur via dropdown des 31 secteurs définis dans `allSectors`, + description), et `_buildStep3` (montant FCFA + zones d'upload pointillées via `DottedBorder` + `_DottedPainter`).

À la validation, on affiche un `SnackBar` vert et on ferme la page. Le pitch n'est pas encore persisté — c'est sur la TODO Livrable 1.

Fichiers : `lib/screens/pitch_page.dart` · `lib/screens/root_shell.dart`

5.11 UX premium

Le splash `SplashPage` utilise un `AnimationController` 1.1 s avec un mixin

`TickerProviderStateMixin`. Les 3 orbites drapeau s'allument en cascade via des `Interval` (0.30→0.50, 0.40→0.60, 0.50→0.70).

Le curseur étoiles est dans `CursorFollower` (web/desktop seulement, no-op via `kIsWeb`). Il maintient une liste de `_Star` avec position, couleur (drapeau Sénégal), durée de vie 900 ms. Un `Ticker` supprime les étoiles expirées et le `CustomPainter` `_StarsPainter` les redessine.

Le hover glow sur les cards est implémenté dans `HoverGlowCard` via `MouseRegion` + `AnimatedContainer` (scale 1.015 + shadow ambré). Toutes les cards interactives passent par ce wrapper : `MentorCard`, `_StatCard`, `_DerCard`.

Les compteurs animés utilisent `AnimatedCounter` (`TweenAnimationBuilder` + `Curves.easeOutQuart` sur 1.1 s). Les transitions de page sont configurées globalement dans `main.dart` via `_FadeThroughBuilder`.

Fichiers : `lib/screens/splash_page.dart` · `lib/widgets/cursor_follower.dart` ·
`lib/widgets/hover_glow_card.dart` · `lib/widgets/animated_counter.dart` ·
`lib/main.dart`

5.12 Branch protection GitHub

Configurée via Settings → Branches sur le repo. La branche `main` exige : Pull Request avec 1 approval (owner), dismissal des stale approvals, conversation resolution avant merge, et bloque les force pushes et deletions. Pour le développeur, le workflow est : `git checkout -b feat/...` → `git push -u origin feat/...` → `gh pr create` → review → merge.

6. Reste à faire

Périmètre découpé par échéance. Les TODO les plus urgents sont en haut. Le code couleur du tableau : rouge = bloquant pour la prochaine livraison, ambre = important, gris = nice to have.

6.1 Court terme — Livrable 1

Tâche	Priorité	Notes
Inscription Mentor + Investisseur	HAUTE	Champs spécifiques (expertise, ticket d'investissement, vérification CIS)
Dashboard Mentor (vue dédiée)	HAUTE	Mes mentees, demandes en attente, sessions à venir, mes pitches reçus
Dashboard Investisseur (vue dédiée)	HAUTE	Pitches à étudier, portefeuille, ROI, mes investissements en cours
Vraie messagerie temps réel	HAUTE	Page conversations + chat. Firebase Realtime DB ou Firestore selon volume
Dépôt de pitch persistant	HAUTE	Stocker dans /pitches/{pitchId}, lier à user.projects, upload PDF via Storage
Upload photo de profil	MOYENNE	Firebase Storage + remplacer Avatar initials par CachedNetworkImage
Système de favoris fonctionnel	MOYENNE	Toggle bookmark sur mentor → /users/{uid}/favorites
Réservation de session mentor	MOYENNE	Page calendrier réelle + node /sessions avec slot, confirmation
Notifications push (FCM)	MOYENNE	Firebase Cloud Messaging — token par device + topic par rôle
Mot de passe oublié	HAUTE	FirebaseAuth.sendPasswordResetEmail() + page basique de saisie d'email
Vérification de l'email à l'inscription	HAUTE	user.sendEmailVerification() — banner non-vérifié + reauth après clic sur le lien
OTP par téléphone	HAUTE	Phone Auth Firebase (le doc DIAPALER original le mentionne explicitement)
Suppression de compte (CDP Sénégal)	HAUTE	Obligation légale RGPD/CDP — supprime user Firebase Auth + node /users/{uid}

Tâche	Priorité	Notes
Édition / suppression d'un projet existant	MOYENNE	Aujourd'hui on peut créer mais pas modifier — ajouter EditProjectPage
Module DER/FJ avec vraies fiches	MOYENNE	Fonctionnalité F7 du doc cours — fiches PAVIE 2 + Be Yes + critères d'éligibilité
Vue calendrier des sessions	MOYENNE	Sessions à venir + passées côté entrepreneur (et côté mentor au Livrable 2)
Notation et avis post-session	MOYENNE	Fonctionnalité F11 du doc — alimente le compatibility score du matching
Bibliothèque de ressources	BASSE	Fonctionnalité F10 — articles, mini-formations, modèles (BP, étude marché) FR/wolof
Mode sombre (réactiver)	BASSE	Code de AppTheme.dark déjà présent — juste rebrancher le toggle

6.2 Moyen terme — Livrable 2

- Configuration Android (google-services.json + plugin Gradle) pour APK
- Configuration iOS (GoogleService-Info.plist) si soumission App Store
- Déploiement Firebase Hosting pour la version web publique
- CI/CD GitHub Actions : flutter analyze + flutter test + build à chaque PR
- Couverture de tests > 50 % (widgets + unit tests sur services)
- Internationalisation FR / Wolof avec flutter_intl
- Mode hors-ligne partiel (cache Hive ou Isar pour mentors et conversations)
- Page DER/FJ avec vraies fiches PAVIE 2 + Be Yes + check d'éligibilité

6.3 Long terme — Vision Livrable 3+

- Algorithme de matching avancé (scoring pondéré multi-critères ou ML léger)
- Intégration mobile money pour paiement de session (Wave / Orange Money / Free Money)
- Vidéoconférence intégrée (Jitsi Meet ou Daily.co)
- Système de notation/avis post-session (alimente le compatibility score)
- Analytics (Firebase Analytics + dashboard métier interne)
- Recherche full-text sur mentors via Algolia ou Typesense
- Webhook Calendly / Google Calendar pour synchro des créneaux
- Export PDF du pitch deck à partager hors plateforme
- Programme de parrainage (lien d'invitation + récompense au filleul actif)

7. Workflow équipe

7.1 Conventions de commit

Utiliser un préfixe pour clarifier la nature du commit :

Préfixe	Pour quoi
feat:	Nouvelle fonctionnalité
fix:	Correction de bug
ux:	Amélioration UI/UX (style, animation)
perf:	Optimisation performance
refactor:	Refonte de code sans nouvelle fonctionnalité
docs:	Documentation (README, guides)
chore:	Tâches techniques (deps, gitignore, etc.)
test:	Ajout / modif de tests

7.2 Workflow Pull Request

```
# 1. Créer une branche feature à partir de main git checkout main && git pull git checkout -b feat/nom-de-la-fonctionnalite # 2. Coder, committer, pusher git add lib/... git commit -m "feat: description courte" git push -u origin feat/nom-de-la-fonctionnalite # 3. Ouvrir une PR sur GitHub vers main gh pr create --base main --title "feat: ..." --body "..." # 4. Attendre la review de l'owner # Si changements demandés : push de nouveaux commits sur la même branche # 5. Une fois approuvée, merger via l'UI GitHub (Squash and merge recommandé)
```

7.3 Avant chaque commit

- flutter analyze — doit retourner « No issues found »
- Vérifier visuellement que l'app tourne en mode release sans erreur console
- Si modification de UserProfile : penser à updater DatabaseService._toMap / _fromMap
- Pas de print() ni de TODO commit dans le code de la PR

8. Points d'attention

⚠ Sécurité Firebase

Les règles Realtime Database sont actuellement en mode test (lecture/écriture ouvertes 30 jours). Avant tout déploiement public, il faut écrire les règles réelles : un user peut lire/écrire seulement /users/{uid} où {uid} == auth.uid.

□ Plateforme

Pour l'instant l'app ne tourne que sur Chrome (web). La config Android/iOS Firebase n'est pas faite — c'est ~30 min de boulot mais à planifier avant de pouvoir distribuer un APK ou TestFlight.

□ Performance

Le mode --debug est très lent en web (5× plus lent que release). Pour la démo, toujours utiliser --release. Pour le développement, utiliser le hot reload avec --debug malgré la lenteur — ça vaut le coup pour itérer.

9. Ressources

- Repo GitHub : github.com/Souleymane-Sirima-Mbodj/Diapaler-Africa
- Console Firebase : console.firebase.google.com/project/diapaler-africa
- Documentation Flutter : docs.flutter.dev
- FlutterFire (plugin Firebase) : firebase.flutter.dev
- Guide d'installation : docs/Guide_Installation_DIAPALER.docx
- Doc fonctionnelle (Livrable 0 du cours) : DOCX original de l'équipe
- Maquettes haute fidélité : présentation PowerPoint du Livrable 0

« *Connecte ton idée à ton succès* »

— L'équipe DIAPALER AFRICA —

Document généré pour les développeurs · Bon code □□